Flask for Full-Stack Engineers

From Zero to L1 & L2 Interview Ready

A complete, hands-on guide covering Python & web fundamentals, the Flask framework, templates, databases, REST APIs, authentication, testing, deployment, full-stack integration, and 80+ curated interview questions with answers for Associate Software Engineer roles.

Module 1 — Foundations: Web, HTTP & Python Refresher

1.1 How the Web Works

When a user types a URL into the browser, the browser sends an **HTTP request** to a server. The server runs application code (in our case, a Flask app), prepares a response, and returns it as **HTML, JSON, or other content**. The browser then renders that response. This request-response cycle is the foundation of every web application.

1.2 HTTP Essentials

- **Methods:** GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
- **Status codes:** 1xx informational, 2xx success (200 OK, 201 Created), 3xx redirect, 4xx client error (400, 401, 403, 404), 5xx server error (500, 502, 503).
- **Headers:** metadata such as Content-Type, Authorization, Cookie, Accept.
- **Body:** payload of the request/response, typically JSON, form data, or HTML.
- **Stateless:** each request is independent; state is kept via cookies, sessions, or tokens.

1.3 Python Refresher (must-know before Flask)

Flask is written in Python, so a working command of these concepts is essential:

- **Data types & structures:** str, int, float, list, dict, tuple, set.
- **Control flow:** if/elif/else, for, while, break, continue.
- **Functions:** def, *args, **kwargs, default arguments, lambdas.
- **OOP:** classes, __init__, inheritance, dunder methods.
- **Modules & packages:** import, from … import, __init__.py.
- **Decorators:** functions that wrap other functions — Flask routing uses them heavily.
- **Virtual environments:** python -m venv venv, isolating dependencies.

Decorator quick refresher

```
def greet_decorator(func):
    def wrapper(*args, **kwargs):
        print("Hello before calling function")
        result = func(*args, **kwargs)
        print("Bye after calling function")
        return result
    return wrapper

@greet_decorator
def say(name):
    print(f"Hi {name}")

say("Riya")
# Output:
# Hello before calling function
# Hi Riya
# Bye after calling function
```

Note: *Flask's* ***@app.route("/")*** *is a decorator that registers your function as a request handler.*

Module 2 — Flask: Setup, Project Structure & First App

2.1 What is Flask?

Flask is a **micro web framework** for Python created by Armin Ronacher. It's called "micro" because it ships with a small core (routing, templating via Jinja2, request handling) and lets you choose libraries for the rest (ORM, forms, auth). It is built on **Werkzeug** (WSGI utilities) and **Jinja2** (templating).

2.2 Flask vs Django (interview favourite)

Aspect	Flask	Django
Type	Micro / minimalist	Full-featured / batteries-included
ORM	Choose your own (SQLAlchemy)	Built-in Django ORM
Admin panel	Not included	Built-in
Learning curve	Easy, gradual	Steeper at first
Best for	APIs, microservices, small-to-mid apps	Large monolithic apps, CMS
Flexibility	Very high	Opinionated

2.3 Installation

```
# 1. Create a project folder
mkdir flask_app && cd flask_app

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate         # macOS / Linux
venv\Scripts\activate            # Windows

# 3. Install Flask
pip install flask

# 4. Verify
python -c "import flask; print(flask.__version__)"
```

2.4 Your First Flask App

```
# app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, Flask!"

if __name__ == "__main__":
    app.run(debug=True)
```

Run it:

```
python app.py
# or
flask --app app run --debug
```

Visit **http://127.0.0.1:5000/** in the browser. You should see **Hello, Flask!**

2.5 Recommended Project Structure

```
flask_app/
■■■ venv/
■■■ app/
■   ■■■ __init__.py        # application factory
■   ■■■ routes.py          # or use blueprints
■   ■■■ models.py          # SQLAlchemy models
■   ■■■ forms.py
■   ■■■ templates/
■   ■   ■■■ index.html
■   ■■■ static/
■       ■■■ css/
■       ■■■ js/
■■■ tests/
■■■ config.py
■■■ requirements.txt
■■■ run.py
```

Module 3 — Routing, Requests & Responses

3.1 Basic Routing

```
@app.route("/about")
def about():
    return "About page"

@app.route("/contact")
def contact():
    return "Contact us"
```

3.2 Dynamic URLs (URL Variables)

```
@app.route("/user/<username>")
def show_user(username):
    return f"Profile of {username}"

@app.route("/post/<int:post_id>")
def show_post(post_id):
    return f"Post number {post_id}"
```

Converters: **string** (default), **int**, **float**, **path**, **uuid**.

3.3 HTTP Methods

```
from flask import request

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        return f"Logged in as {username}"
    return "<form method='POST'>...</form>"
```

3.4 The Request Object

- **request.args** — query string params (?key=value)
- **request.form** — form data (POST)
- **request.json** — parsed JSON body
- **request.files** — uploaded files
- **request.headers** — request headers
- **request.cookies** — cookies sent by client
- **request.method** — GET / POST / PUT…

3.5 Returning Different Response Types

```
from flask import jsonify, redirect, url_for, make_response

@app.route("/api/user")
def api_user():
    return jsonify({"id": 1, "name": "Riya"})

@app.route("/old")
def old():
    return redirect(url_for("home"))

@app.route("/custom")
def custom():
    resp = make_response("Custom response", 201)
```

```
    resp.headers["X-Custom"] = "Hello"
    return resp
```

3.6 url_for — Always Generate URLs This Way

Hard-coding URLs is fragile. Use **url_for(endpoint_name, **params)** so links automatically update if the route changes.

```
url_for("show_user", username="riya")   # -> /user/riya
url_for("static", filename="css/style.css")  # -> /static/css/style.css
```

Module 4 — Templates with Jinja2

4.1 Why Templates?

Returning raw strings won't get you far. Jinja2 lets you write HTML files with placeholders, loops, and conditionals — keeping logic separate from presentation.

4.2 Rendering a Template

```
from flask import render_template

@app.route("/hello/<name>")
def hello(name):
    return render_template("hello.html", name=name, items=["A","B","C"])
```

templates/hello.html:

```
<!doctype html>
<html>
  <body>
    <h1>Hello, {{ name }}!</h1>
    <ul>
      {% for item in items %}
        <li>{{ item }}</li>
      {% endfor %}
    </ul>
    {% if name == "admin" %}
      <p>Welcome back, boss.</p>
    {% endif %}
  </body>
</html>
```

4.3 Template Inheritance

Define a base layout once, extend it elsewhere. This is one of the most powerful Jinja2 features.

```
<!-- templates/base.html -->
<html>
<head><title>{% block title %}My Site{% endblock %}</title></head>
<body>
  <nav>...</nav>
  <main>{% block content %}{% endblock %}</main>
  <footer>© 2026</footer>
</body>
</html>

<!-- templates/home.html -->
{% extends "base.html" %}
{% block title %}Home{% endblock %}
{% block content %}
  <h1>Welcome</h1>
{% endblock %}
```

4.4 Filters & Built-ins

- **{{ name|upper }}** — uppercase
- **{{ price|round(2) }}** — round to 2 decimals
- **{{ items|length }}** — list length
- **{{ html|safe }}** — render raw HTML (be careful — XSS risk)
- **{{ date|default('N/A') }}** — fallback if undefined

4.5 Auto-escaping & Security

Jinja2 auto-escapes variables, which prevents **XSS attacks**. Only use the **|safe** filter when you completely trust the content.

Module 5 — Forms, Validation & File Uploads

5.1 Raw HTML Form Handling

```
@app.route("/submit", methods=["GET", "POST"])
def submit():
    if request.method == "POST":
        email = request.form.get("email")
        if not email or "@" not in email:
            return "Invalid email", 400
        return f"Saved {email}"
    return render_template("form.html")
```

5.2 Flask-WTF (Recommended)

Flask-WTF integrates WTForms with Flask: clean validation, CSRF protection, and templating helpers.

```
pip install flask-wtf
```

```
# forms.py
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Email, Length

class LoginForm(FlaskForm):
    email    = StringField("Email", validators=[DataRequired(), Email()])
    password = PasswordField("Password", validators=[DataRequired(), Length(min=6)])
    submit   = SubmitField("Login")

# app.py
app.config["SECRET_KEY"] = "change-me"

@app.route("/login", methods=["GET", "POST"])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        return f"Welcome {form.email.data}"
    return render_template("login.html", form=form)
```

```
<!-- login.html -->
<form method="POST">
  {{ form.hidden_tag() }}      {# CSRF token #}
  {{ form.email.label }} {{ form.email() }}
  {{ form.password.label }} {{ form.password() }}
  {{ form.submit() }}
</form>
```

5.3 File Uploads

```
import os
from werkzeug.utils import secure_filename

app.config["UPLOAD_FOLDER"] = "uploads"
ALLOWED = {"png", "jpg", "jpeg", "pdf"}

def allowed(filename):
    return "." in filename and filename.rsplit(".",1)[1].lower() in ALLOWED

@app.route("/upload", methods=["POST"])
def upload():
    f = request.files["file"]
    if f and allowed(f.filename):
        name = secure_filename(f.filename)
```

```
        f.save(os.path.join(app.config["UPLOAD_FOLDER"], name))
        return "Uploaded"
    return "Invalid file", 400
```

Note: *Always use* ***secure_filename()*** *— it strips path traversal attacks like ../../etc/passwd.*

Module 6 — Static Files & Frontend Integration

6.1 Serving Static Files

Place CSS, JS, images in the **static/** folder. Reference with **url_for('static', filename=…)**.

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
<script src="{{ url_for('static', filename='js/app.js') }}"></script>
<img src="{{ url_for('static', filename='img/logo.png') }}" alt="logo">
```

6.2 Full-Stack Architectures with Flask

(a) Server-Rendered (Monolith)

Flask returns full HTML pages via Jinja2. Best for traditional CRUD apps, dashboards, internal tools.

(b) Flask as API + Separate Frontend (SPA)

Flask exposes a JSON REST API. The frontend (React, Vue, Angular) is a separate app that consumes it. This is the standard for modern full-stack projects.

6.3 Enabling CORS for SPA Frontends

```
pip install flask-cors
```

```
from flask_cors import CORS
CORS(app, resources={r"/api/*": {"origins": "http://localhost:3000"}})
```

6.4 Communicating from React (fetch)

```
// React component
useEffect(() => {
  fetch("http://localhost:5000/api/users")
    .then(r => r.json())
    .then(setUsers);
}, []);
```

Module 7 — Databases with Flask-SQLAlchemy

7.1 Setup

```
pip install flask-sqlalchemy
```

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///site.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db = SQLAlchemy(app)
```

7.2 Defining Models

```
class User(db.Model):
    id       = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    email    = db.Column(db.String(120), unique=True, nullable=False)
    posts    = db.relationship("Post", backref="author", lazy=True)

class Post(db.Model):
    id      = db.Column(db.Integer, primary_key=True)
    title   = db.Column(db.String(200), nullable=False)
    content = db.Column(db.Text, nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False)
```

7.3 Creating Tables

```
with app.app_context():
    db.create_all()
```

7.4 CRUD Operations

```
# CREATE
u = User(username="riya", email="riya@example.com")
db.session.add(u)
db.session.commit()

# READ
all_users = User.query.all()
one       = User.query.filter_by(username="riya").first()
by_id     = User.query.get(1)

# UPDATE
one.email = "new@example.com"
db.session.commit()

# DELETE
db.session.delete(one)
db.session.commit()
```

7.5 Querying — Common Patterns

```
User.query.filter(User.email.like("%@gmail.com")).all()
User.query.order_by(User.username.desc()).limit(10).all()
db.session.query(User).join(Post).filter(Post.title.contains("Flask")).all()
```

7.6 Relationships Cheat Sheet

- **One-to-Many:** *db.relationship* + ForeignKey (User → many Posts).

- **Many-to-Many:** use an association table.
- **One-to-One:** add *uselist=False* on the relationship.

Module 8 — Migrations with Flask-Migrate

db.create_all() is fine in dev but breaks down when you change schema in production. **Flask-Migrate** (wraps Alembic) tracks schema changes with versioned migration files.

```
pip install flask-migrate
```

```
from flask_migrate import Migrate
migrate = Migrate(app, db)
```

```
# CLI workflow
flask db init                 # one time
flask db migrate -m "add users"
flask db upgrade              # apply
flask db downgrade            # revert
```

Module 9 — Authentication, Sessions & JWT

9.1 Sessions

Flask stores small bits of user data in client-side, signed (not encrypted) cookies.

```
from flask import session

app.config["SECRET_KEY"] = "super-secret"

@app.route("/login", methods=["POST"])
def login():
    session["user"] = request.form["username"]
    return "Logged in"

@app.route("/logout")
def logout():
    session.pop("user", None)
    return "Logged out"
```

9.2 Password Hashing (NEVER store plain text!)

```
from werkzeug.security import generate_password_hash, check_password_hash

hashed = generate_password_hash("mypassword")
check_password_hash(hashed, "mypassword")   # True
```

9.3 Flask-Login

```
pip install flask-login
```

```
from flask_login import (LoginManager, UserMixin, login_user,
                         logout_user, login_required, current_user)

login_manager = LoginManager(app)
login_manager.login_view = "login"

class User(db.Model, UserMixin):
    id       = db.Column(db.Integer, primary_key=True)
    email    = db.Column(db.String(120), unique=True)
    password = db.Column(db.String(200))

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route("/dashboard")
@login_required
def dashboard():
    return f"Hello {current_user.email}"
```

9.4 JWT Authentication (for APIs)

Session cookies don't suit SPAs and mobile clients well. **JWT** (JSON Web Tokens) are the standard for stateless API auth.

```
pip install flask-jwt-extended
```

```
from flask_jwt_extended import (JWTManager, create_access_token,
                                jwt_required, get_jwt_identity)

app.config["JWT_SECRET_KEY"] = "jwt-super-secret"
jwt = JWTManager(app)
```

```
@app.route("/api/login", methods=["POST"])
def api_login():
    data = request.json
    # ... verify user ...
    token = create_access_token(identity=data["email"])
    return jsonify(access_token=token)

@app.route("/api/me")
@jwt_required()
def me():
    return jsonify(user=get_jwt_identity())
```

Module 10 — Building REST APIs

10.1 RESTful Conventions

Method	Endpoint	Purpose
GET	/api/users	List all users
GET	/api/users/<id>	Retrieve one user
POST	/api/users	Create a new user
PUT	/api/users/<id>	Replace user fully
PATCH	/api/users/<id>	Update fields
DELETE	/api/users/<id>	Delete user

10.2 Building a CRUD API

```
@app.route("/api/users", methods=["GET"])
def list_users():
    users = User.query.all()
    return jsonify([{"id": u.id, "email": u.email} for u in users])

@app.route("/api/users", methods=["POST"])
def create_user():
    data = request.json
    if not data or "email" not in data:
        return jsonify(error="email required"), 400
    u = User(email=data["email"])
    db.session.add(u); db.session.commit()
    return jsonify(id=u.id, email=u.email), 201

@app.route("/api/users/<int:uid>", methods=["GET"])
def get_user(uid):
    u = User.query.get_or_404(uid)
    return jsonify(id=u.id, email=u.email)

@app.route("/api/users/<int:uid>", methods=["PUT"])
def update_user(uid):
    u = User.query.get_or_404(uid)
    u.email = request.json["email"]
    db.session.commit()
    return jsonify(id=u.id, email=u.email)

@app.route("/api/users/<int:uid>", methods=["DELETE"])
def delete_user(uid):
    u = User.query.get_or_404(uid)
    db.session.delete(u); db.session.commit()
    return "", 204
```

10.3 Flask-RESTful & Marshmallow

For larger APIs, use **Flask-RESTful** (class-based resources) or **Flask-Smorest**. Use **Marshmallow** for serialization & input validation:

```
from marshmallow import Schema, fields, validate

class UserSchema(Schema):
    id    = fields.Int(dump_only=True)
    email = fields.Email(required=True)
```

```
    name  = fields.Str(validate=validate.Length(min=2))
```

Module 11 — Error Handling & Logging

11.1 Custom Error Pages

```
@app.errorhandler(404)
def not_found(e):
    return render_template("404.html"), 404

@app.errorhandler(500)
def server_error(e):
    return render_template("500.html"), 500
```

11.2 Raising HTTP Errors

```
from flask import abort
abort(403, description="You don't have access")
```

11.3 Logging

```
import logging
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

app.logger.info("App started")
app.logger.error("Something went wrong")
```

11.4 Try / Except Pattern in API Handlers

```
@app.route("/api/safe")
def safe():
    try:
        # risky work
        result = do_something()
        return jsonify(result=result)
    except ValueError as e:
        return jsonify(error=str(e)), 400
    except Exception:
        app.logger.exception("Unexpected error")
        return jsonify(error="internal"), 500
```

Module 12 — Blueprints & Application Factory

12.1 Why Blueprints?

As an app grows, putting every route in **app.py** becomes unmaintainable. **Blueprints** let you split your app into logical modules (auth, blog, admin, api…).

12.2 Defining a Blueprint

```
# app/auth/routes.py
from flask import Blueprint, render_template

auth_bp = Blueprint("auth", __name__, url_prefix="/auth")

@auth_bp.route("/login")
def login():
    return render_template("auth/login.html")
```

12.3 Application Factory Pattern

```
# app/__init__.py
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

def create_app(config_class="config.DevConfig"):
    app = Flask(__name__)
    app.config.from_object(config_class)

    db.init_app(app)

    from app.auth.routes import auth_bp
    from app.main.routes import main_bp
    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)

    return app
```

```
# run.py
from app import create_app
app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

> ***Note:*** *This pattern is essential for testing (you create separate app instances per config) and is what most production codebases use.*

Module 13 — Testing Flask Apps (pytest)

13.1 Setup

```
pip install pytest
```

13.2 Test Fixture

```
# tests/conftest.py
import pytest
from app import create_app, db

@pytest.fixture
def client():
    app = create_app("config.TestConfig")
    with app.test_client() as client:
        with app.app_context():
            db.create_all()
        yield client
```

13.3 Writing Tests

```
def test_home(client):
    resp = client.get("/")
    assert resp.status_code == 200
    assert b"Hello" in resp.data

def test_create_user(client):
    resp = client.post("/api/users", json={"email": "a@b.com"})
    assert resp.status_code == 201
    assert resp.get_json()["email"] == "a@b.com"

def test_user_not_found(client):
    resp = client.get("/api/users/9999")
    assert resp.status_code == 404
```

13.4 Run Tests

```
pytest -v
pytest --cov=app tests/        # with coverage
```

Module 14 — Deployment: Gunicorn, Docker & Cloud

14.1 Why Not Flask's Built-in Server?

app.run() uses Werkzeug's dev server — single-threaded, unstable under load, not production-safe. Use a real **WSGI server** like **Gunicorn** or **uWSGI**.

14.2 Gunicorn

```
pip install gunicorn
gunicorn -w 4 -b 0.0.0.0:8000 "run:app"
# -w 4   four worker processes
# -b     bind to host:port
```

14.3 Typical Production Stack

- **Nginx** — reverse proxy / static files / TLS termination
- **Gunicorn** — WSGI server running your Flask app
- **PostgreSQL** — production database
- **Redis** — caching, sessions, background queues
- **Celery** — async tasks (emails, reports, etc.)

14.4 Dockerfile Example

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

EXPOSE 8000
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8000", "run:app"]
```

14.5 docker-compose.yml

```
version: "3.9"
services:
  web:
    build: .
    ports: ["8000:8000"]
    environment:
      - DATABASE_URL=postgresql://app:app@db/app
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

14.6 Cloud Options

- **Render / Railway / Fly.io** — simplest, push from Git.
- **Heroku** — classic PaaS.
- **AWS** — EC2 + RDS + ALB, or Elastic Beanstalk, or ECS/Fargate.

- **GCP** — Cloud Run (containerized Flask), App Engine.
- **Azure** — App Service.

14.7 Production Checklist

- Set **DEBUG=False**.
- Keep **SECRET_KEY** in environment variables — never commit.
- Use HTTPS (TLS) everywhere.
- Run migrations on deploy (**flask db upgrade**).
- Configure structured logging & error monitoring (Sentry).
- Use a reverse proxy (Nginx) in front of Gunicorn.
- Cache static files; consider a CDN.

Module 15 — Full-Stack Project: Notes App

Bring it all together. Build a small full-stack notes application — exactly the kind of project interviewers expect an Associate Software Engineer to ship.

15.1 Features

- User registration & login (Flask-Login or JWT)
- Create / Read / Update / Delete notes
- Each note belongs to a user
- REST API + optional React frontend
- PostgreSQL in production, SQLite in dev
- Tested with pytest, deployed via Docker

15.2 Models

```
class User(db.Model, UserMixin):
    id       = db.Column(db.Integer, primary_key=True)
    email    = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)
    notes    = db.relationship("Note", backref="owner", lazy=True,
                               cascade="all, delete-orphan")

class Note(db.Model):
    id      = db.Column(db.Integer, primary_key=True)
    title   = db.Column(db.String(200), nullable=False)
    content = db.Column(db.Text)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False)
    created = db.Column(db.DateTime, default=datetime.utcnow)
```

15.3 Core API Routes

```
POST   /api/register          -> create account
POST   /api/login             -> get JWT
GET    /api/notes             -> list current user's notes
POST   /api/notes             -> create
GET    /api/notes/<id>        -> retrieve
PUT    /api/notes/<id>        -> update
DELETE /api/notes/<id>        -> delete
```

15.4 Sample Endpoint

```
@app.route("/api/notes", methods=["POST"])
@jwt_required()
def create_note():
    data = request.json
    user_email = get_jwt_identity()
    user = User.query.filter_by(email=user_email).first()
    note = Note(title=data["title"], content=data.get("content",""), owner=user)
    db.session.add(note); db.session.commit()
    return jsonify(id=note.id, title=note.title), 201
```

15.5 Suggested Additions to Talk About in Interviews

- Pagination (*?page=2&per_page=10*).
- Search (*?q=keyword*).
- Rate limiting (Flask-Limiter).
- Caching frequent reads with Flask-Caching + Redis.

- Background tasks with Celery (e.g. send welcome email).

Module 16 — L1 Interview Questions (Basics)

Typical screening / first-round questions for Associate / Junior full-stack roles.

Q1. What is Flask?

A: Flask is a lightweight Python micro web framework built on Werkzeug (WSGI) and Jinja2 (templating). It provides routing, request handling, and templating out of the box, while leaving DB, auth, and form libraries to your choice.

Q2. Why is Flask called a micro-framework?

A: Because its core is intentionally small. It doesn't force a database, ORM, form library, or auth system. You add only what you need via extensions.

Q3. How is Flask different from Django?

A: Flask is minimalist and flexible; Django is full-featured (admin panel, ORM, auth built in). Flask suits APIs and small-to-mid apps; Django suits large monolithic apps with many built-ins.

Q4. What is WSGI?

A: Web Server Gateway Interface — the Python standard (PEP 3333) for how web servers (Gunicorn, uWSGI) talk to Python web apps. Flask is a WSGI application.

Q5. What is Jinja2?

A: The templating engine Flask uses to render HTML with placeholders, loops, conditionals, filters, and inheritance. It auto-escapes variables to help prevent XSS.

Q6. Explain @app.route().

A: It's a decorator that maps a URL pattern to a view function. When a request matches the URL, Flask calls that function and uses its return value as the response.

Q7. What is the request object?

A: A global proxy in Flask that holds data about the incoming HTTP request — form data, query params, JSON body, headers, files, cookies, method.

Q8. How do you return JSON?

A: Use **flask.jsonify(data)**. It serialises Python dicts/lists to JSON and sets the correct Content-Type header.

Q9. What is url_for() and why use it?

A: It generates a URL for a given view function (by endpoint name). Using it instead of hard-coded URLs means your links stay valid even if you change the route path.

Q10. What does debug mode do?

A: Enables the interactive debugger, auto-reloads code on changes, and shows tracebacks in the browser. Never use it in production — the debugger lets anyone execute arbitrary Python.

Q11. How are sessions implemented in Flask?

A: Flask stores session data in a cryptographically signed cookie on the client. The data isn't encrypted by default — it's tamper-proof, not secret. A SECRET_KEY is required.

Q12. What is the difference between session and cookie?

A: A cookie is any key-value pair stored in the browser. A session (in Flask) is a specific signed cookie that the framework uses to track per-user data securely.

Q13. How do you serve static files?

A: Put them in the **static/** folder, reference via **url_for('static', filename='css/style.css')**.

Q14. What is a Blueprint?

A: A way to group related routes, templates, and static files into a module that can be registered on the app. Used to organise large apps.

Q15. What is the application context vs the request context?

A: App context holds app-level globals (current_app, g). Request context holds request-level globals (request, session). Flask pushes both for each request.

Q16. How do you handle form data?

A: Set **methods=['POST']**, then read **request.form['field_name']**. For validation/CSRF, use Flask-WTF.

Q17. What is CSRF and how does Flask-WTF handle it?

A: Cross-Site Request Forgery — a malicious site tricks the user's browser into sending requests to your site. Flask-WTF adds a hidden CSRF token to every form and validates it on submit.

Q18. How does Flask-SQLAlchemy fit in?

A: It's a Flask extension that integrates SQLAlchemy ORM with Flask — handles app context, session lifecycle, and config conveniently.

Q19. How do you connect Flask to a database?

A: Install Flask-SQLAlchemy, set **SQLALCHEMY_DATABASE_URI**, define models inheriting from **db.Model**, and use **db.session** to commit changes.

Q20. How do you create tables?

A: db.create_all() in development. In production, use Flask-Migrate (Alembic) with **flask db migrate** and **flask db upgrade**.

Q21. What's the purpose of SECRET_KEY?

A: Used by Flask to sign session cookies and CSRF tokens. Without it, sessions and forms won't work securely. Keep it in env vars, never commit.

Q22. Difference between GET and POST?

A: GET retrieves data (idempotent, parameters in URL, cacheable). POST submits data (non-idempotent, body, not cached by default).

Q23. What is REST?

A: An architectural style using HTTP verbs (GET, POST, PUT, DELETE), stateless requests, and resource-based URLs (/users/1/posts/3) to build APIs.

Q24. How do you return a 404?

A: Use **abort(404)** or **return 'Not found', 404**. Use **get_or_404()** on queries for cleaner code.

Q25. What does jsonify do that json.dumps doesn't?

A: Sets the correct **Content-Type: application/json** header, supports Flask config like JSON_SORT_KEYS, and is the idiomatic way to return JSON.

Q26. How do you redirect?

A: from flask import redirect, url_for; return redirect(url_for('home'))

Q27. What's the role of werkzeug?

A: It's the WSGI utility library Flask is built on. It provides request/response objects, routing, secure_filename, password hashing helpers, and the dev server.

Q28. How do you upload a file?

A: Use a form with **enctype='multipart/form-data'**, in Flask read **request.files['file']** and call **secure_filename()** before saving.

Q29. How to enable CORS?

A: Install flask-cors and call **CORS(app)** or apply per blueprint with origin restrictions. Needed when your SPA frontend is on a different domain/port.

Q30. What is the difference between PUT and PATCH?

A: PUT replaces the whole resource. PATCH applies a partial update. Both are idempotent in their semantics.

Module 17 — L2 Interview Questions (Intermediate & Advanced)

Expect these in the second round, technical screens with engineers/leads.

Q1. Explain Flask's application factory pattern. Why use it?

A: A function (typically **create_app(config)**) that builds and returns a Flask app. It lets you create multiple isolated app instances with different configs — essential for tests, for using extensions cleanly, and for running CLI/scripts with their own context.

Q2. Explain app context vs request context in detail.

A: Both are stacks managed by Flask. The **application context** binds **current_app** and **g** (a scratch object for the request) so extensions know which app is active. The **request context** binds **request** and **session**. A request context implicitly pushes an app context. Outside a request (scripts, tests), you push them manually using **with app.app_context():**.

Q3. What is g and when do you use it?

A: flask.g is a per-request scratch namespace. Common use: store the current authenticated user or a DB connection at the start of a request and reuse it across helpers and templates during that request.

Q4. How does Flask handle concurrent requests in production?

A: Flask itself is single-threaded, but in production you run multiple workers/threads via Gunicorn or uWSGI. Workers handle requests concurrently. For high I/O, use async workers like **gevent** or **eventlet**.

Q5. How do Blueprints help with scaling code?

A: They modularise routes, templates, static files, and error handlers. Each feature lives in its own folder. You can apply URL prefixes, subdomains, and CLI commands per blueprint. They also enable circular-import-safe structures when paired with the application factory.

Q6. Explain Flask-SQLAlchemy session lifecycle.

A: db.session is a scoped session tied to the request. It holds added/changed objects in memory until you call **commit()**. Flask-SQLAlchemy hooks into request teardown to remove the session and return connections to the pool, preventing leaks.

Q7. How do you optimise SQLAlchemy queries (N+1 problem)?

A: The N+1 problem happens when you query a list, then access a related attribute per row triggering N more queries. Fix with **joinedload()** or **selectinload()** in **options(...)** to eager-load relationships. Use indexes on frequently filtered columns, limit columns with **load_only()**, and paginate.

Q8. Difference between lazy='select', 'joined', 'subquery', 'dynamic' in SQLAlchemy relationships?

A: select (default): load on access via separate query. **joined**: load via SQL JOIN in the parent query. **subquery**: load via a separate subquery. **dynamic**: returns a query object so you can filter before executing — best for very large collections.

Q9. How would you handle DB migrations safely in production?

A: Use Flask-Migrate (Alembic). Write reversible migrations. For zero-downtime: add new columns/tables first, deploy code that writes to both old and new, backfill, switch reads, then drop old. Always back up before running destructive migrations.

Q10. How do you implement JWT auth and what are the trade-offs?

A: Use Flask-JWT-Extended. The server issues a signed token on login; clients send it in the Authorization header. Pros: stateless, scales horizontally, good for SPAs/mobile. Cons: tokens can't be revoked easily (mitigate with short expiry + refresh tokens, or a denylist in Redis); larger than a session cookie.

Q11. Compare session-based auth vs JWT.

A: Sessions: server-side state via signed cookie ID, easy to revoke, CSRF risk. JWT: stateless, scales easily, fits SPAs/mobile, harder to revoke, susceptible to XSS if stored in localStorage. Most production apps use sessions for browser flows and JWT for APIs/mobile.

Q12. How to secure a Flask app — major items?

A: HTTPS everywhere, secure SECRET_KEY in env, hash passwords with werkzeug or bcrypt/argon2, enable CSRF on forms, set HttpOnly/Secure/SameSite cookies, validate & sanitise input, parameterise queries (ORM does this), rate-limit endpoints, set security headers (Content-Security-Policy, X-Frame-Options), audit dependencies.

Q13. How do you implement rate limiting?

A: Use Flask-Limiter with Redis as a backend. Apply default limits app-wide and per-route limits on sensitive endpoints (login, signup, password-reset). Use a key function tied to user or IP.

Q14. How do you handle long-running tasks?

A: Offload to a background queue. Typical stack: **Celery** workers backed by **Redis** or **RabbitMQ**. The HTTP request enqueues a task and returns immediately; the worker picks it up and runs it. You can poll or push status to clients (e.g. WebSockets, Server-Sent Events).

Q15. How would you cache responses?

A: Use Flask-Caching with Redis. Apply **@cache.cached(timeout=60)** on read-heavy view functions or **@cache.memoize()** on helpers. Invalidate keys on writes. Use HTTP cache headers (ETag, Cache-Control) for client/CDN caching of static API responses.

Q16. How to test a Flask app?

A: Use pytest with the test client. Create a **TestConfig** with an in-memory SQLite DB and **TESTING=True**. Use fixtures to build app/client per test. Cover routes, models, auth flows, error paths. Aim for high coverage on critical paths; measure with **pytest-cov**.

Q17. What is Werkzeug's role in Flask?

A: Werkzeug provides the WSGI fundamentals: request/response objects, routing system (URL maps and converters), the dev server, password hashing utilities, the debugger, and helpers like **secure_filename**. Flask is essentially a thin layer on top of Werkzeug + Jinja2.

Q18. Explain how Flask routes URLs internally.

A: Werkzeug builds a URL map of all routes. On each request, it matches the path against the map, extracts URL variables (with converters like int, uuid), and dispatches to the bound view function. Endpoints (the route names) decouple view function references from path strings.

Q19. What is middleware in Flask? How do you write one?

A: Flask doesn't use the term 'middleware' the way Django does, but you can use WSGI middleware (wrap **app.wsgi_app**) or use **@app.before_request** / **@app.after_request** / **@app.teardown_request** hooks for cross-cutting concerns like auth checks, logging, response headers.

Q20. How to handle environment-specific config?

A: Define **DevConfig**, **TestConfig**, **ProdConfig** classes. Choose one via env var: **app.config.from_object(os.getenv('CONFIG','config.DevConfig'))**. Keep secrets in env vars or a secret manager — never commit them.

Q21. Difference between flask run and gunicorn?

A: flask run uses Werkzeug's dev server — single-threaded, with debugger and reloader. **Gunicorn** is a production WSGI server: multi-worker, robust, with sync/async worker classes. Always use Gunicorn (or uWSGI) in production behind Nginx.

Q22. How to scale a Flask app?

A: Vertical: more CPU/RAM, more Gunicorn workers (rule of thumb: 2×cores+1). Horizontal: multiple app instances behind a load balancer. Externalise state — DB, Redis for sessions/cache, S3 for files. Add read replicas, use caching layers, CDN for static assets, async workers for heavy jobs.

Q23. How does Flask handle thread safety?

A: Each request has its own context. **request**, **g**, **session** are context-local proxies, so they are safe across threads. Avoid module-level mutable state. Database sessions are scoped per request and removed at teardown.

Q24. How would you design a multi-tenant Flask app?

A: Options: schema-per-tenant in PostgreSQL, row-level isolation via a **tenant_id** column with a query-level filter (e.g. via a SQLAlchemy event), or database-per-tenant. Pick a tenant from subdomain, header, or token, stash it in **g.tenant** in **before_request**, and enforce filtering everywhere.

Q25. How to version a REST API?

A: URL prefix (**/api/v1/users**) is the most common and explicit. Alternatives: Accept header versioning, or query parameter. Use blueprints per version to keep code separated, deprecate old versions on a schedule.

Q26. How to handle file uploads at scale?

A: Don't store on local disk in a multi-instance deploy. Upload directly to object storage (S3) using presigned URLs, store only the URL/key in your DB. Validate type, size, and scan for malware as needed.

Q27. Explain pagination strategies.

A: Offset/limit (*?page=2&per_page=20*): simple, but slow on deep pages and unstable when data changes. **Cursor/keyset** pagination (*?after=<id>*): stable and fast for ordered data. SQLAlchemy offers **paginate()** for offset pagination out of the box.

Q28. How do you debug a slow endpoint in production?

A: Add structured logs with timing, use APM tools (New Relic, Datadog, Sentry Performance), enable SQLAlchemy query logging, check slow query logs in the DB, profile locally with **cProfile**, look for N+1 queries, missing indexes, large serialisation, blocking I/O.

Q29. What is gunicorn's preload option?

A: --preload loads the app once in the master process before forking workers, which reduces memory via copy-on-write and speeds up boot. Downside: code changes need a full restart and you must avoid resources (DB connections, file handles) opened pre-fork.

Q30. How do you do dependency injection in Flask?

A: Flask favours simple patterns: extensions stored on **app**, request-scoped values on **g**, factories for services. For larger codebases, use lightweight DI libs (e.g. **dependency-injector**) to pass

repositories/services into views and keep them testable.

Q31. Explain the WSGI lifecycle of a request in Flask.

A: Server receives the request → calls **app.wsgi_app(environ, start_response)** → Flask builds Request and Response contexts → matches URL → runs **before_request** hooks → calls view function → runs **after_request** hooks → returns Response → server sends bytes → **teardown_request** / **teardown_appcontext** fire (closing DB sessions etc.).

Q32. How would you implement WebSockets in Flask?

A: Flask is synchronous, so use **Flask-SocketIO** on top of an async server like **eventlet** or **gevent**. For pure async, consider **Quart** (Flask-API-compatible async framework).

Q33. When would you NOT pick Flask?

A: When you need a heavy built-in admin and tight coupling out of the box → Django is faster to deliver. When you need first-class async at scale → FastAPI or Quart. When you need very high-throughput type-safe APIs with auto-docs → FastAPI.

Module 18 — Coding Round Problems

Tasks Associate-level full-stack candidates are commonly asked to implement live or as a take-home.

Problem 1 — Build a Mini URL Shortener

Requirements: POST a long URL, get back a short code; GET / redirects.

```
import string, random
from flask import Flask, request, redirect, jsonify, abort

app = Flask(__name__)
store = {}  # in memory; in real life use Redis/DB

def gen_code(n=6):
    return "".join(random.choices(string.ascii_letters + string.digits, k=n))

@app.route("/shorten", methods=["POST"])
def shorten():
    url = request.json.get("url")
    if not url:
        return jsonify(error="url required"), 400
    code = gen_code()
    while code in store:
        code = gen_code()
    store[code] = url
    return jsonify(short=f"/{code}")

@app.route("/<code>")
def follow(code):
    url = store.get(code)
    if not url:
        abort(404)
    return redirect(url)
```

Problem 2 — Todo REST API

Implement CRUD for todos with title and done flag, persisted via SQLAlchemy.

```
class Todo(db.Model):
    id    = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    done  = db.Column(db.Boolean, default=False)

@app.route("/todos", methods=["GET"])
def list_todos():
    return jsonify([{"id":t.id,"title":t.title,"done":t.done} for t in Todo.query.all()])

@app.route("/todos", methods=["POST"])
def add_todo():
    data = request.json
    t = Todo(title=data["title"])
    db.session.add(t); db.session.commit()
    return jsonify(id=t.id), 201

@app.route("/todos/<int:tid>", methods=["PATCH"])
def toggle(tid):
    t = Todo.query.get_or_404(tid)
    t.done = request.json.get("done", t.done)
    db.session.commit()
    return jsonify(done=t.done)

@app.route("/todos/<int:tid>", methods=["DELETE"])
def delete(tid):
```

```
    t = Todo.query.get_or_404(tid)
    db.session.delete(t); db.session.commit()
    return "", 204
```

Problem 3 — Implement a Login & Protected Route

```
@app.route("/api/register", methods=["POST"])
def register():
    data = request.json
    if User.query.filter_by(email=data["email"]).first():
        return jsonify(error="email taken"), 400
    user = User(email=data["email"],
                password=generate_password_hash(data["password"]))
    db.session.add(user); db.session.commit()
    return jsonify(id=user.id), 201

@app.route("/api/login", methods=["POST"])
def login():
    data = request.json
    user = User.query.filter_by(email=data["email"]).first()
    if not user or not check_password_hash(user.password, data["password"]):
        return jsonify(error="bad credentials"), 401
    token = create_access_token(identity=user.email)
    return jsonify(token=token)

@app.route("/api/me")
@jwt_required()
def me():
    return jsonify(email=get_jwt_identity())
```

Problem 4 — Pagination

```
@app.route("/api/posts")
def posts():
    page     = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 10, type=int)
    p = Post.query.order_by(Post.id.desc()).paginate(page=page, per_page=per_page)
    return jsonify(
        items=[{"id":x.id,"title":x.title} for x in p.items],
        total=p.total, pages=p.pages, page=p.page
    )
```

Problem 5 — Rate Limiting Login

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(get_remote_address, app=app, default_limits=["100/hour"])

@app.route("/api/login", methods=["POST"])
@limiter.limit("5/minute")
def login():
    ...
```

Module 19 — Cheat Sheet & Final Tips

19.1 Flask CLI Quick Reference

```
flask --app run run --debug      # dev server
flask routes                     # list all routes
flask shell                      # interactive shell with app context
flask db init / migrate / upgrade  # Flask-Migrate
gunicorn -w 4 run:app            # production
```

19.2 Common Imports

```
from flask import (Flask, request, jsonify, render_template,
                   redirect, url_for, session, g, abort, make_response,
                   Blueprint, current_app)
from flask_sqlalchemy import SQLAlchemy
from flask_migrate    import Migrate
from flask_login      import LoginManager, login_user, logout_user, login_required, current_user,
from flask_wtf        import FlaskForm
from flask_jwt_extended import JWTManager, create_access_token, jwt_required, get_jwt_identity
from flask_cors       import CORS
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils   import secure_filename
```

19.3 Status Code Quick Memory

- **200 OK** · **201 Created** · **204 No Content**
- **301 Moved Permanently** · **302 Found** · **304 Not Modified**
- **400 Bad Request** · **401 Unauthorized** · **403 Forbidden** · **404 Not Found** · **409 Conflict** · **422 Unprocessable** · **429 Too Many Requests**
- **500 Internal Server Error** · **502 Bad Gateway** · **503 Service Unavailable**

19.4 Interview Day Tips

- Walk through a project you actually built — own it, know every file.
- Always mention **application factory + blueprints** when asked about structure.
- Never say "I'd use db.create_all() in production" — say **Flask-Migrate**.
- Mention **Gunicorn behind Nginx** when asked about deployment.
- When asked "why Flask?" — say flexibility, microservice-friendly, easy to learn, huge ecosystem.
- If stuck on a coding problem: clarify, write the data model first, then the routes, then the validation.
- Talk about **security** proactively: hashing, CSRF, HTTPS, env vars for secrets, input validation.
- Be ready to discuss the **N+1 problem** and how you fixed it.

19.5 30-Day Learning Plan

- **Days 1–3:** Python refresher + Flask Hello World + routing.
- **Days 4–6:** Templates + forms + static files.
- **Days 7–10:** SQLAlchemy + Flask-Migrate + CRUD.
- **Days 11–14:** Auth (sessions + JWT) + Flask-Login.
- **Days 15–18:** REST API design + Marshmallow + Postman testing.
- **Days 19–21:** Blueprints + app factory + error handling + logging.

- **Days 22–24:** Testing with pytest + coverage.
- **Days 25–27:** Deployment with Gunicorn + Docker + Render/Railway.
- **Days 28–30:** Build the Notes app end-to-end + revise interview Q&A.

All the best for your interviews! Build, break, rebuild — that's how you turn this material into instinct.